

PyTorch 图像分类面试模板修改指南

基于你上传的 transfer_learning_template.py / code_template_solid.py 分类任务模板整理

核心结论

如果面试中给你一个 Data 数据集，让你完成猫/狗、车辆/飞机、缺陷/正常等图像分类任务，你当前模板是合适的。你主要需要会灵活修改：数据路径、训练参数、是否冻结 backbone、单张预测图片路径，以及在特殊数据结构下修改 dataloader。

- 最常见情况：数据已经按 ImageFolder 结构组织，直接修改 DATA_DIR 即可。
- 时间紧/CPU 环境：设置 FEATURE_EXTRACT=True，只训练最后分类层。
- 有 GPU/数据较多：可以设置 FEATURE_EXTRACT=False，全模型 fine-tuning。
- 核心模型修改：替换 ResNet18 的最后 fc 层，使输出维度等于当前类别数。

1. 首先判断任务类型：分类，不是检测

如果每张图片只有一个类别标签，例如 cat、dog、car、plane，这就是图像分类任务。分类任务不需要 bounding box，只需要 image 和 label。你的模板使用 datasets.ImageFolder，因此最适合下面这种文件夹结构：

```
Data/
├── train/
│   ├── cat/
│   │   ├── cat_001.jpg
│   │   └── cat_002.jpg
│   └── dog/
│       ├── dog_001.jpg
│       └── dog_002.jpg
└── val/
    ├── cat/
    │   └── cat_101.jpg
    └── dog/
        └── dog_101.jpg
```

ImageFolder 会根据子文件夹名自动识别类别名，例如 class_names = ["cat", "dog"]。因此类别数可以自动使用 len(class_names)，不需要手动写死。

2. 必改位置一：DATA_DIR

原始模板中：

```
DATA_DIR = "hymenoptera_data"
```

面试时如果数据集文件夹叫 Data，直接改成：

```
DATA_DIR = "Data"
```

然后快速检查数据结构：

```
print(os.listdir(DATA_DIR))
print(os.listdir(os.path.join(DATA_DIR, "train")))
print(os.listdir(os.path.join(DATA_DIR, "val")))
```

你要确认 Data 下面有 train 和 val，且 train/val 下面的类别文件夹一致。

3. 必改位置二：训练参数

模板中的主要参数包括：

```
NUM_EPOCHS = 25
BATCH_SIZE = 8
NUM_WORKERS = 0
```

```
LEARNING_RATE = 0.001
MOMENTUM = 0.9
STEP_SIZE = 7
GAMMA = 0.1
FEATURE_EXTRACT = False
```

面试现场推荐默认配置：

```
NUM_EPOCHS = 3
BATCH_SIZE = 8
NUM_WORKERS = 0
LEARNING_RATE = 0.001
MOMENTUM = 0.9
STEP_SIZE = 2
GAMMA = 0.1
FEATURE_EXTRACT = True
```

原因：训练更快、CPU 也能跑、不容易过拟合，并且足够展示完整 pipeline。如果有 GPU 且数据集较大，可以改为 FEATURE_EXTRACT=False、NUM_EPOCHS=5、BATCH_SIZE=16。

4. 必懂位置三：FEATURE_EXTRACT 的含义

模板中：

```
FEATURE_EXTRACT = False
# False: fine-tune the whole ResNet18
# True: freeze backbone and train only the final classifier
```

FEATURE_EXTRACT=True 表示冻结 ResNet18 的 backbone，只训练最后新替换的 fc 层。适合小数据集、CPU、时间紧的面试场景。

FEATURE_EXTRACT=False 表示微调整个 ResNet18。适合有 GPU、数据较多、希望提高最终性能的情况。

你可以这样解释：

Since the interview time is limited and the dataset may be small, I freeze the pretrained backbone and only train the final classifier. This makes training faster and reduces overfitting.

5. 核心模型修改：替换 ResNet18 最后一层

模板的 build_model() 是分类任务核心：

```
def build_model(num_classes, feature_extract=False):
    model = models.resnet18(weights="IMAGENET1K_V1")

    set_parameter_requires_grad(model, feature_extract)

    num_ftrs = model.fc.in_features
    model.fc = nn.Linear(num_ftrs, num_classes)

    return model
```

原始 ResNet18 是 ImageNet 1000 分类器。这里保留预训练特征提取能力，只把最后 fully connected layer 替换成当前任务的类别数量。

```
num_classes = len(class_names)
model.fc = nn.Linear(num_ftrs, num_classes)
```

6. 数据读取：ImageFolder 自动识别类别

模板中的 build_dataloaders() 会自动读取 train/val，并根据文件夹名称得到类别：

```
image_datasets = {
    phase: datasets.ImageFolder(
        root=os.path.join(data_dir, phase),
        transform=data_transforms[phase]
    )
    for phase in ["train", "val"]
}
```

```
}

class_names = image_datasets["train"].classes
```

因此只要文件夹命名正确，类别名和类别数会自动获得。你不需要手动指定 NUM_CLASSES。

7. 单张图片预测路径需要修改

原始模板最后的路径是 ants/bees 示例：

```
test_img_path = os.path.join(
    DATA_DIR,
    "val",
    "bees",
    "72100438_73de9f17af.jpg"
)
```

如果是猫狗分类，改成实际图片路径：

```
test_img_path = os.path.join(
    DATA_DIR,
    "val",
    "cat",
    "cat_001.jpg"
)
```

不知道图片名时，先打印：

```
print(os.listdir(os.path.join(DATA_DIR, "val")))
print(os.listdir(os.path.join(DATA_DIR, "val", class_names[0]))[:5])
```

8. 如果没有 val 文件夹怎么办？

有些面试数据只有：

```
Data/
├── cat/
└── dog/
```

这时不能直接用当前 build_dataloaders()，因为模板默认需要 Data/train 和 Data/val。你可以使用 random_split 自动划分：

```
from torch.utils.data import random_split

def build_dataloaders_auto_split(data_dir, batch_size, num_workers, train_ratio=0.8):
    data_transforms = get_data_transforms()

    full_dataset = datasets.ImageFolder(
        root=data_dir,
        transform=data_transforms["train"]
    )

    class_names = full_dataset.classes
    train_size = int(train_ratio * len(full_dataset))
    val_size = len(full_dataset) - train_size

    train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])
    val_dataset.dataset.transform = data_transforms["val"]

    dataloaders = {
        "train": torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,
            shuffle=True, num_workers=num_workers),
        "val": torch.utils.data.DataLoader(val_dataset, batch_size=batch_size,
            shuffle=False, num_workers=num_workers),
    }
    dataset_sizes = {"train": len(train_dataset), "val": len(val_dataset)}
    return dataloaders, dataset_sizes, class_names, data_transforms
```

注意：如果数据已经分好 train/val，就不要用自动切分版本。

9. 如果标签在 CSV 文件里怎么办？

如果数据是 images 文件夹 + labels.csv，而不是 ImageFolder 结构，就需要写自定义 Dataset：

```
import pandas as pd
from torch.utils.data import Dataset

class CSVDataset(Dataset):
    def __init__(self, csv_path, img_dir, transform=None):
        self.df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.transform = transform
        self.class_names = sorted(self.df["label"].unique())
        self.class_to_idx = {c: i for i, c in enumerate(self.class_names)}

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img_path = os.path.join(self.img_dir, row["filename"])
        img = Image.open(img_path).convert("RGB")
        label = self.class_to_idx[row["label"]]
        if self.transform is not None:
            img = self.transform(img)
        return img, label
```

但普通面试分类题大概率会给你 ImageFolder 结构，因为它最适合快速完成 PyTorch 分类任务。

10. 哪些部分通常不用改？

- get_device()：自动选择 CUDA 或 CPU。
- train_model()：已经包含 train/eval 切换、forward、loss、backward、optimizer.step、validation 和保存最佳 checkpoint。
- plot_history()：自动画 loss 和 accuracy 曲线。
- visualize_model()：显示验证集图片，展示预测标签和真实标签。
- predict_single_image()：函数本身不用改，只改图片路径。

11. 面试可直接使用的 plan

I will solve this as an image classification problem using transfer learning. First, I will inspect the dataset structure and check whether it follows the ImageFolder format with train and validation folders. Then I will define ImageNet-style transforms, load data using ImageFolder, and infer class names automatically. Next, I will use a pretrained ResNet18 model and replace its final fully connected layer according to the number of target classes. If the dataset is small or only CPU is available, I will freeze the backbone and train only the final classifier. Otherwise, I can fine-tune the whole model. I will train using cross-entropy loss, SGD optimizer, and a learning-rate scheduler. Finally, I will evaluate validation accuracy, plot training curves, save the best checkpoint, and run inference on validation images.

12. 面试常见问题回答

Q: Why use pretrained ResNet18?

A: Because it already contains strong visual features learned from ImageNet. Transfer learning is faster and more stable than training from scratch on a small dataset.

Q: Why replace the final fc layer?

A: The original ResNet18 outputs 1000 ImageNet classes. Our task has a different number of classes, so the final layer must be replaced.

Q: Why use CrossEntropyLoss?

A: It is the standard loss for single-label multi-class classification and combines softmax with negative log likelihood.

Q: Why use different transforms for train and val?

A: Training uses random augmentation to improve generalization. Validation uses deterministic preprocessing for stable evaluation.

13. 最终修改清单

修改项	建议值 / 操作	原因
DATA_DIR	"Data"	指向面试给定数据集
NUM_EPOCHS	3	快速跑通流程
BATCH_SIZE	4 或 8	CPU/Jupyter 更稳
NUM_WORKERS	0	避免 Windows/Jupyter 多进程问题
FEATURE_EXTRACT	True	冻结 backbone, 加快训练
test_img_path	实际验证集图片路径	用于单图预测展示

最终建议：面试中默认使用 FEATURE_EXTRACT=True、NUM_EPOCHS=3、NUM_WORKERS=0，先保证完整 pipeline 可以跑通。如果时间和 GPU 允许，再尝试全模型 fine-tuning。